Ch6 函式與遞迴

6-1 函式

- 函式用於結構化程式
- 將相同功能的程式獨立出來，經由函式的呼叫，傳入資料與回傳處理後的結果
- 程式設計師只要將函式寫好，就可以不斷利用此函式做相同動作，同時可使程式碼不重複，而若要修改此功能，只要更改此函式。

- 其他程式設計師要使用此函式，只要知道此函式的功能，什麼輸入會有怎樣的輸出，而不需知道函式實作的細節。
- 函式可幫助多位程式設計師共同開發系統，事先規劃好函式名稱與功能，再各自開發函式與整合所有程式，最後達成系統所需求的功能。

6-1-1　函式的定義、傳回值與呼叫

- 自訂函式需要包含兩個部分，分別是「函式的定義」與「函式的呼叫」。
  - 「函式的定義」是實作函式的功能，輸入參數與回傳處理後的結果
  - 「函式的呼叫」是其他程式中呼叫自訂函式，讓自訂函式真正執行

6-1-1　函式的定義、傳回值與呼叫

- **函式的定義**
  - 以def 開頭，空一個空白字元(space)，接函式名稱後，串接著一對小括號，小括號可以填入要傳入函式的參數
  - 當參數有多個的時候以逗號隔開，右小括號後面須接上「:」，函式範圍以縮行固定個數的空白字元表示，縮行相同個數的空白字元的程式碼就是函式的作用範圍。

6-1-1 函式的定義、傳回值與呼叫

– 當函式需要傳回值使用指令return，表示函式回傳資料給原呼叫函式，若不需要回傳值的函式就不需要加上return，函式的定義與傳回值格式，如下表。

表 6-1 函式的定義

分類	函式的定義語法	範例
不回傳值的函式	def 函式名稱 (參數 1 , 參數 2 , …): 　函式的敘述區塊	def hi(): 　print('hi')

回傳值的函式	def 函式名稱 (參數 1 , 參數 2 , …):
	　函式的敘述區塊
	　return 要傳回的變數或值
		def min(a,b):
		　if a > b:
		　　return b
		　else:
		　　return a

6-1-1　函式的定義、傳回值與呼叫

- **函式的呼叫**
  - 程式經由函式呼叫，將資料傳入函式，函式處理後傳回結果給呼叫程式，程式中如何呼叫函式？在程式中利用函式名稱與參數來呼叫函式。

– 等號右邊要先做完，利用函式名稱與參數來呼叫函式，最後函式回傳值給變數，變數就紀錄函式呼叫後的回傳值。

方法一：不回傳值函式的呼叫語法

```
函式名稱 ( 參數值 1, 參數值 2,…)
```

方法二：回傳值函式的呼叫語法

```
變數 = 函式名稱 ( 參數值 1, 參數值 2,…)
```

6-1-1 函式的定義、傳回值與呼叫

綜合前面敘述，函式定義與函式呼叫範例 ( ：ch6\6-1-1-func1.py)，如下。

行號	範例	執行結果
1	def hi():	
2	print('hi')	
3	hi()	
4	def min(a,b):	
5	if a > b:	hi
6	return b	2
7	else:	
8	return a	
9	print(min(2,4))	

程式解說

- 第1 到2 行：自訂函式hi，印出「hi」到螢幕上。
- 第3 行：呼叫hi 函式。
- 第4 到8 行：自訂函式min，輸入兩個參數a 與b，將較小的數字回傳回來。
- 第9 行：呼叫函式min，輸入數字2 與4，回傳較小的數字2，使用函式print 將數字2 顯示到螢幕上。

6-1-1　函式的定義、傳回值與呼叫

- 以下範例用於計算長方形面積，使用者輸入長度與寬度，呼叫自訂的函式area 將長度與寬度傳入，回傳計算結果。

行號	函式範例程式 ( ：ch6\ 6-1-1-rec.py)	執行結果
1	def area(x,y):	
2	return x*y	
3	a = int(input(' 請輸入長度？ '))	請輸入長度？ 3
4	b = int(input(' 請輸入寬度？ '))	請輸入寬度？ 4
5	ans = area(a, b)	長方形面積爲 12
6	print(' 長方形面積爲 ', ans)	

6-1-1　函式的定義、傳回值與呼叫

- 程式解說
  - 第1 行到第2 行：函式area 的定義，輸入參數x與y，回傳x 乘以y 的結果。
  - 第3 行：於螢幕輸出「請輸入長度？」，經由函式input 輸入字串，再由函式int 將字串轉換成整數物件，變數a參考到此整數物件。

- 第4 行：於螢幕輸出「請輸入寬度？」，經由函式input 輸入字串，再由函式int 將字串轉換成整數物件，變數b參考到此整數物件。
- 第5 行：呼叫area 函式，使用a 與b 為參數，變數ans參考到此函式回傳值。
- 第6 行：使用函式print 顯示字串「長方形面積為」串接變數ans在螢幕上。

6-1-1　函式的定義、傳回值與呼叫

- **函式呼叫過程的流程圖**
  - 執行過程中呼叫函式area 以變數a 與變數b 為輸入值，此時程式執行控制權由原呼叫函式轉換到函式area，函式area 過程中遇到return 將值回傳，此時程式執行控制權由函式area 轉回原呼叫函式。

- 函式呼叫過程的流程圖

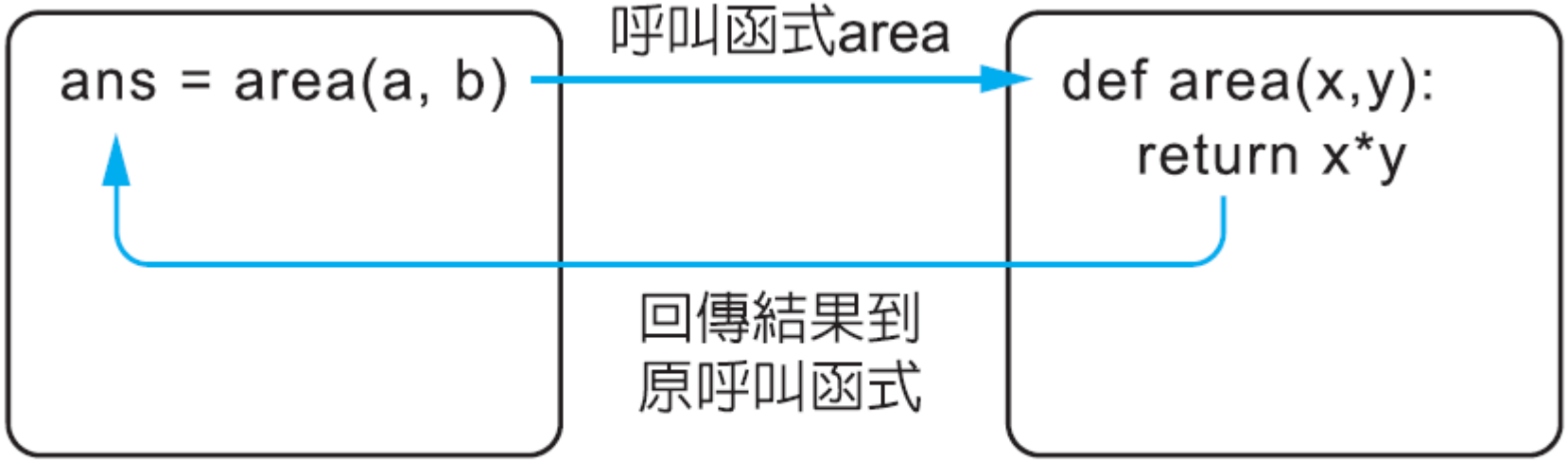


圖 6-1 函式呼叫過程的流程圖

6-1-2 函式與變數的作用範圍

- 變數作用範圍分成全域變數與函式內的區域變數，宣告在最上面最外層的稱作全域變數，宣告在函式內的變數稱作區域變數，函式內若沒有那個變數就會往函式外找尋，舉例如以下範例(ch6\6-1-2a-func2.py)。

行號	範例	執行結果
1 2 3 4	g = 5 def f1(): print(g) f1()	5

程式解說

- 第3 行的print(g)，因為函式內沒有變數g，所以往函式外去找尋，找到第1 行的變數g，將全域變數g 的數值5 顯示在螢幕上。

- 若函式內有一個區域變數，與全域變數名稱一樣的變數，若讀取區域變數在初始化區域變數之前，則會產生 UnboundLocalError 錯誤

- 下表中程式第3 行，因為會產生 UnboundLocalError 錯誤，所以使用井字號「#」進行註解讓該行沒有作用，若要測試此錯誤就可以將井字號「#」刪除，再執行程式一次，就會出現UnboundLocalError錯誤。

行號	範例	執行結果

1
2
3
4
5
6
7
	g = 5
	def f2():
	  #print(g)
	  g = 10
	  print(g)
	f2()
	print(g)
		10
		5

- 全域變數g 與區域變數g，是兩個不同的變數
- 函式內區域變數g 作用範圍在函式內，全域變數g 作用範圍為整個檔案，但因為函式內區域變數有相同的變數名稱，函式會優先使用區域變數，若找不到才去找全域變數。

- 程式解說
  - **第1 行**：宣告全域變數g，初始化為5。
  - **第2到5行**：定義函式f2，宣告區域變數g，並初始化為10(第4行)，印出區域變數g 的值到螢幕上(第5 行)。
  - **第6 行**：呼叫函式f2。
  - **第7 行**：印出全域變數g 的值到螢幕上。

函式內若沒有那個變數就會往函式外去找尋，也可以使用 global 宣告區域變數，該區域變數將明確指向全域變數，也就是宣告爲 global 的區域變數一定指向相同名稱的全域變數，舉例如以下範例 ( ：ch6\6-1-2b-func3.py)。

行號	範例	執行結果
1 2 3 4 5 6 7 8	g = 5 def f(): global g print(g) g = 10 print(g) f() print(g)	5 10 10

6-1-2 函式與變數的作用範圍

- 程式解說
  - **第1 行**：宣告全域變數g，初始化為5。
  - **第2 到6 行**：定義函式f，宣告g 為全域變數(第3 行)，印出全域變數g 的值到螢幕上(第4 行)，設定全域變數g 為10(第5 行)，印出全域變數g 的值到螢幕上(第6 行)。
  - **第7 行**：呼叫函式f。
  - **第8 行**：印出全域變數g 的值到螢幕上。

6-2　函式範例練習

- 範例6-2-1 計算BMI (ch6\6-2-1-BMI.py)
  - BMI 常用來判斷肥胖程度，BMI 等於體重(KG)除以身高(M) 的平方，「BMI 與肥胖等級標準」表，如下。請寫一個程式讓使用者輸入體重與身高，顯示BMI 值與肥胖程度。

表 6-2　BMI 值

BMI 值	肥胖等級
BMI ＜ 18	體重過輕
18 ≦ BMI ＜ 24	體重正常
24 ≦ BMI ＜ 27	體重過重
27 ≦ BMI	體重肥胖

範例6-2-1 計算BMI (ch6\6-2-1-BMI.py)

- 解題想法
  - 我們利用自訂函式BMI，輸入體重與身高回傳BMI 值，利用BMI 值與「BMI 與肥胖等級標準」表，使用選擇結構顯示BMI 值與對應的肥胖等級。

範例6-2-1 計算BMI (ch6\6-2-1-BMI.py)

行數	程式碼	執行結果
1	def BMI(w, h):	請輸入體重爲「80」與身高爲「1.7」，顯示計算所得BMI值爲「27.68166089965398」與肥胖分級爲「體重肥胖」。 請輸入體重 (KG) ？ 80 請輸入身高 (M) ？ 1.7 BMI 爲 27.68166089965398 體重肥胖
2	return w/(h*h)	
3	w = float(input(' 請輸入體重 (KG) ？ '))	
4	h = float(input(' 請輸入身高 (M) ？ '))	
5	bmi = BMI(w, h)	
6	print('BMI 爲 ', bmi)	
7	if (bmi < 18):	
8	print(' 體重過輕 ')	
9	elif (bmi < 24):	
10	print(' 體重正常 ')	
11	elif (bmi < 27):	
12	print(' 體重過重 ')	
13	else:	
14	print(' 體重肥胖 ')	

程式解說

- 第1 到2 行：定義BMI 函式，使用體重與身高為輸入值，回傳BMI 值。體重儲存於變數w，身高儲存於變數h，回傳「w/(h*h)」(第2 行)。
- 第3 行：於螢幕輸出「請輸入體重(KG) ？」，經由函式input 輸入字串，再由函式float 將字串轉換成浮點數物件，變數w參考到此浮點數物件。

程式解說

- 第4 行：於螢幕輸出「請輸入身高(M) ？」，經由函式input 輸入字串，再由函式float 將字串轉換成浮點數物件，變數h參考到此浮點數物件。
- 第5 行：呼叫BMI函式，使用w與h 為參數，將所得BMI值指定給變數bmi。

程式解說

- 第6 行：將函式所得的BMI值顯示在螢幕。
- 第7 到8 行：判斷所計算出的BMI值是否小於18，若是則顯示「體重過輕」。

程式解說

- 第9 到10 行：否則判斷所計算出的BMI 值是否小於24(隱含成績大於等於18)，若是則顯示「體重正常」。
- 第11 到12 行：否則判斷所計算出的BMI 值是否小於27(隱含成績大於等於24)，若是則顯示「體重過重」。
- 第13 到14 行：否則(隱含成績大於等於27) 顯示「體重肥胖」。

範例6-2-2 求質數 (ch6\6-2-2-prime.py)

- 某數的因數只有1 與自己，沒有其他因數，稱為質數，寫一個程式列出1 到100 所有質數。

解題想法

- 自訂判斷質數的函式，輸入一個數字，回傳是否為質數，回傳True 表示為質數，回傳False 表示為非質數
- 接著使用迴圈結構列出由1 到100 所有數，將每個數輸入到判斷質數函數，若判斷質數函數回傳True，表示該數為質數，印出該數到螢幕上。

範例6-2-2 求質數 (ch6\6-2-2-prime.py)

行數	程式碼	執行結果
1	import math	程式執行結果如下。 顯示最後 6 個質數。 71 爲質數 73 爲質數 79 爲質數 83 爲質數 89 爲質數 97 爲質數
2	def prime(num):	
3	j = 2	
4	while j <= math.sqrt(num):	
5	if (num%j == 0):	
6	return False	
7	j += 1	
8	return True	
9	for i in range(2, 101):	
10	if prime(i):	
11	print(i, ' 為質數 ')	

程式解說

- 第1 行：匯入模組math。
- 第2 到8 行：自訂函式prime，以整數變數num 為輸入，若num 為質數，回傳True，若num 不為質數，回傳False。j 為整數變數，用於while 迴圈，變數j 依序指向所有小於 $\sqrt{num}$ 的數，初始值為2(第3 行)。

- 當變數j 小於等於 $\sqrt{num}$ (第4 行)，繼續測試變數j 是否可以整除函式的輸入值num(第5 行)，若是，則變數j 為輸入值num 的因數，回傳False(第6 行)，變數j 值加1(第7 行)，重複while 迴圈回到
- 第4 行。若while 迴圈執行完畢，沒有發現因數，則回傳True(第8 行)。

- 第9 到11 行：迴圈變數i，其值變化由2 到100。
- 呼叫prime 函式，prime 函式輸入值為迴圈變數i，若i 為質數，prime 函式回傳True，否則回傳False。
- 使用if 結構判斷函式prime 是否回傳True，若是則顯示變數i 串接「為質數」(第10 到11 行)。

6-3 函式的輸入與輸出

- 6-3-1 函式的輸入
  - 函式中有預設值的輸入參數一定要放在後面，預設值要是不可以變的常數，不能為串列或字典等可以修改的資料結構。

行號	範例 ( ：ch6\6-3-1-func4.py)	執行結果
1 2 3	def f(s, count=1): print(s * count) f('Hi')	Hi

程式解說

- 第1 到2 行：定義函式f，輸入參數有s 與count，參數count 預設值為1，使用print印出count 個字串s 到螢幕(第2 行)。
- 第3 行：呼叫函式f，設定s為「Hi」，變數count沒有輸入值，使用預設值1。

6-3-1　函式的輸入

可以經由函式呼叫輸入新的數值取代原預設值，如以下範例。

行號	範例 (：ch6\6-3-1-func4.py)	執行結果
1	def f(s, count=1):	
2	print(s * count)	HiHiHi
3	f('Hi',3)	

程式解說

- 第1 到2 行：定義函式f，輸入參數有s 與count，參數count 預設值為1，使用print印出count 個字串s 到螢幕(第2 行)。
- 第3 行：呼叫函式f，設定s為「Hi」，設定count為3，取代預設值1。
- 函式內參數的對應，若未指定名稱會依照順序填入，例如：自訂函式func 如下。

6-3-1　函式的輸入

- 函式內參數的對應，若未指定名稱會依照順序填入，例如：自訂函式func 如下。

```
def func(x, y, z=9):
    print("x=" , x , "y=" , y , "z=" , z)
```

- 若以func(1,2) 進行呼叫，會印出「x= 1 y= 2 z= 9」，可以看出依照順序放入對應的參數；
- 若以func(1, 2, 3) 進行呼叫，會印出「x= 1 y= 2 z= 3」，可以看出依照順序放入對應的參數，且預設值被輸入值取代。

- 若以func(x=3, y=4) 進行呼叫，會印出「x= 3 y= 4 z= 9」，可以指定參數與輸入值的對應，這時就可以不用依照順序，
- 若以func(y=5, x=6) 進行呼叫，會印出「x= 6 y= 5 z= 9」，所以指定參數與輸入值的對應，輸入順序與參數的對應順序可以不同。

- 以func(x=3, z=6) 進行呼叫，會發生TypeError，因為參數y 沒有輸入值，沒有預設值的參數一定要有輸入值。

6-3-1 函式的輸入

行號	範例 (：ch6\6-3-1-func5.py)	執行結果
1	def func(x, y, z=9):	
2	print("x=" , x , "y=" , y , "z=" , z)	
3	func(1, 2)	x= 1 y= 2 z= 9
4	func(1, 2, 3)	x= 1 y= 2 z= 3
5	func(x=3, y=4)	x= 3 y= 4 z= 9
6	func(y=5, x=6)	x= 6 y= 5 z= 9
7	#func(x=3, z=6)	

6-3-1　函式的輸入

- 第1 到2 行：定義函式func，輸入參數有x、y 與z，參數預設值為9，使用函式print 印出參數x、y 與的值到螢幕(第2 行)。
- 第3 行：呼叫函式func，依序設定參數x 為1，參數y 為2，參數使用預設值。

- 第4 行：呼叫函式func，依序設定參數x 為1，參數y 為2，參數為3。
- 第5 行：呼叫函式func，使用指定方式設定參數x 為3，參數y 為4，參數使用預設值。

- 第6 行：呼叫函式func，使用指定方式設定參數y 為5，參數x 為6，參數使用預設值。
- 第7 行：呼叫函式func，使用指定方式設定參數x 為3，參數為6，但沒有指定參數y，所以會產生TypeError 錯誤，此行先以井字號「#」開頭當成註解，若要產生TypeError 錯誤，請將開頭的井字號「#」去除，重新執行就會產生TypeError 錯誤。

6-3-2　函式的回傳值

- 函式回傳值可以使用tuple 回傳多個資料，例如：以下ymd 函式使用tuple 回傳時間的年、月與日，模組datetime 的說明請參閱第11 章。
  - def ymd():
  - 　now = datetime.now()
  - 　return (now.year, now.month, now.day)

- 回傳的tuple 可以使用tuple 開箱(tuple uppacking) 取得回傳的多個參數，如下。
  - y, m, d = ymd()

6-3-2 函式的回傳值

行號	範例 (：ch6\6-3-2-func6.py)	執行結果
1	from datetime import datetime	2018 4 30
2	def ymd():	
3	now = datetime.now()	
4	return (now.year, now.month, now.day)	
5	y, m, d = ymd()	
6	print(y,m,d)	

- 第1 行：匯入函式庫datetime的模組datetime。
- 第2到4行：定義函式ymd，設定變數now參考到模組datetime的函式now回傳的物件，使用return 回傳tuple，tuple 內元素依序為目前時間的年(now.year)、目前時間的月(now.month) 與目前時間的日(now.day)。

- 第5 行：呼叫函式ymd，將回傳的tuple，依序放入變數y、m 與d。
- 第6 行：使用函式print 依序顯示變數y、m 與d 的值到螢幕上。

6-3-3 函式的進階輸入 — 位置引數與關鍵字引數

- 位置引數(函式輸入變數的前方使用「*」) 會將函數內多個輸入值群組化成tuple，例如以下範例，慣例使用args 為位置引數名稱，這個變數名稱可以修改成任何變數名稱。
  - def func1(*args):
  - print(' 位置引數為', args)

- 使用「func1(1,2,3)」呼叫函式func1，會印出以下結果。
  - 位置引數為 (1, 2, 3)
- 可以發現args 為tuple，內容為「(1, 2, 3)」。

6-3-3 函式的進階輸入 — 位置引數與關鍵字引數

- 關鍵字引數(函式輸入變數的前方使用「**」) 會將函數內多個輸入值群組化成字典，例如以下範例，慣例使用kwargs 為關鍵字引數名稱，這個變數名稱可以修改成任何變數名稱。
  - def func2(**kwargs):
  - print(‘關鍵字引數為’ , kwargs)

- 使用「func2(a=1, b=2)」呼叫函式func2，會印出以下結果。
  - 關鍵字引數為 {'b': 2, 'a': 1}
- 可以發現kwargs 為字典，內容為「{'b': 2, 'a': 1}」。

6-3-3 函式的進階輸入 — 位置引數與關鍵字引數

- 參數、位置引數與關鍵字引數可以一起使用，如以下範例。
  - def func3(start, *args, **kwargs):
  - print("start=", start)
  - print(" 位置引數為", args)
  - print(" 關鍵字引數為", kwargs)

- 使用「func3(1, 2, 3, a=4, b=5)」呼叫函式func3，會印出以下結果。
  - start= 1
  - 位置引數為 (2, 3)
  - 關鍵字引數為 {'b': 5, 'a': 4}
- 可以發現第一個數字1 指定給start，args為tuple，內容為「(2, 3)」，kwargs 為字典，內容為「{'b': 5, 'a': 4}」

6-3-3 函式的進階輸入 — 位置引數與關鍵字引數

綜合上述範例獲得以下程式。

行號	範例 ( ：ch6\6-3-3-func7.py)	執行結果
1	def func1(*args):	
2	print(' 位置引數爲 ', args)	
3	func1(1,2,3)	
4	def func2(**kwargs):	位置引數爲 (1, 2, 3)
5	print(' 關鍵字引數爲 ', kwargs)	關鍵字引數爲 {'b': 2, 'a': 1}
6	func2(a=1, b=2)	start= 1
7	def func3(start, *args, **kwargs):	位置引數爲 (2, 3)
8	print("start=", start)	關鍵字引數爲 {'b': 5, 'a': 4}
9	print(" 位置引數爲 ", args)	
10	print(" 關鍵字引數爲 ", kwargs)	
11	func3(1, 2, 3, a=4, b=5)	

程式解說

- 第1 到2 行：定義函式func1，允許將所有輸入值轉成位置引數，顯示位置引數在螢幕上(第2 行)。
- 第3 行：呼叫函式func1，使用「1,2,3」為輸入。

- 第4 到5 行：定義函式func2，允許將所有輸入值轉換成關鍵字引數，顯示關鍵字引數在螢幕上(第5 行)。
- 第6 行：呼叫函式func2，使用「a=1, b=2」為輸入。

- 第7 到10 行：定義函式func3，允許將所有函式輸入值轉換成參數、位置引數與關鍵字引數，顯示參數(第8 行)、位置引數(第9 行) 與關鍵字引數(第10 行) 在螢幕上。
- 第11 行：呼叫函式func3，使用「1, 2, 3, a=4, b=5」為輸入。

6-4 函式的說明文件

- 可以在函式下方使用「'''」撰寫函式的說明文件，說明文件可以跨好幾行，直到找到下一個「'''」，使用「'''」會保留第2 行以後所有開頭的空格，如以下範例。

```
def min(a, b):
    ''' 使用 min 可以找出 a 與 b 較小的值
    Args:
        a: 輸入的第一個參數
        b: 輸入的第二個參數
    Returns:
        回傳 a 與 b 中較小的值
    '''
    if a > b:
        return b
    else:
        return a
```

6-4 函式的說明文件

- 使用「help(min)」可以讀取函式的說明文件，如下。

```
Help on function min in module __main__:

min(a, b)
    使用 min 可以找出 a 與 b 較小的值
    Args:
        a: 輸入的第一個參數
        b: 輸入的第二個參數

    Returns:
        回傳 a 與 b 中較小的值
```

6-4 函式的說明文件

- 使用「print(min.__doc__)」可以讀取函式的說明文件，如下

```
使用 min 可以找出 a 與 b 較小的值
    Args:
        a: 輸入的第一個參數
        b: 輸入的第二個參數

    Returns:
        回傳 a 與 b 中較小的值
```

6-4 函式的說明文件

- 我們可以善用函式的說明文件，讓後續維護程式的程式設計師可以快速了解函式的用途與功能。

行號	範例 (: ch6\6-4-func8.py)	執行結果
1	def min(a, b):	Help on function min in module __main__
2	''' 使用 min 可以找出 a 與 b 較小的值	min(a, b)
3	Args:	使用 min 可以找出 a 與 b 較小的值
4	a: 輸入的第一個參數	Args:
5	b: 輸入的第二個參數	a: 輸入的第一個參數
6		b: 輸入的第二個參數
7	Returns:	
8	回傳 a 與 b 中較小的值	Returns:
9	'''	回傳 a 與 b 中較小的值
10	if a > b:	
11	return b	使用 min 可以找出 a 與 b 較小的值
12	else:	Args:
13	return a	a: 輸入的第一個參數
14	help(min)	b: 輸入的第二個參數
15	print(min.__doc__)	
		Returns:
		回傳 a 與 b 中較小的值

程式解說

- 第1 到13 行：定義函式min，函式說明在第2 到9 行。若a 大於b，則回傳b，否則回傳a(第10 到13 行)。
- 第14 行：使用函式help，讀取函式min 的說明文件。
- 第15 行：使用函式print 印出函式min 的說明文件，函式的說明文件屬性為(__doc__)。

6-5 函式視為物件

- Python 中函式視為物件，以函式名稱當成物件，函式名稱加上() 才會執行該函式，範例如下。

```
def add(a, b):
    return a + b
def run(func, x, y):
    return func(x, y)
k = run(add, 10, 20)
print('k=', k)
```

- 函式run 的第1 個參數func 為函式物件，使用func() 呼叫執行func 所指定的函式
- run 函式以「run(add, 10, 20)」執行，則func 會使用函式物件add 取代，run 函式中的func(x,y) 相當於add(x,y)
- 可以看出Python 把函式名稱當成物件使用，上述程式執行結果如下。

6-5 函式視為物件

行號	範例 (：ch6\6-5-func9.py)	執行結果
1	def add(a, b):	k= 30
2	return a + b	
3	def run(func, x, y):	
4	return func(x, y)	
5	k = run(add, 10, 20)	
6	print('k=', k)	

程式解說

- 第1 到2 行：定義函式add，輸入兩個參數a 與b，回傳a 加上b 的結果。
- 第3 到4 行：定義函式run，輸入三個參數func、x 與y，func 需傳入函式物件，函式func 傳入兩個參數x 與y，回傳函式func 的結果。

- 第5 行：使用「run(add, 10, 20)」相當於執行add(10, 20)，回傳結果指定給變數k。
- 第6 行：使用函式print 顯示「k= 」與變數k 的值到螢幕上。

行號	範例 (：ch6\6-5-func9.py)	執行結果
1	def add(a, b):	k= 30
2	return a + b	
3	def run(func, x, y):	
4	return func(x, y)	
5	k = run(add, 10, 20)	
6	print('k=', k)	

程式解說

- 第1 到2 行：定義函式add，輸入兩個參數a 與b，回傳a 加上b 的結果。
- 第3 到4 行：定義函式run，輸入三個參數func、x 與y，func 需傳入函式物件，函式func 傳入兩個參數x 與y，回傳函式func 的結果。

- 第5 行：使用「run(add, 10, 20)」相當於執行add(10, 20)，回傳結果指定給變數k。
- 第6 行：使用函式print 顯示「k= 」與變數k 的值到螢幕上。

6-6 函式lambda

- 函式若只有一行，可以轉換成為函式lambda，函式lambda 的轉換格式如下。
  - lambda 輸入的參數: 函式的定義

原始函式	轉換為 lambda
def add(a, b): return a + b	lambda a, b: a+b

6-6　函式lambda

- 使用函式lambda 重新改寫6-5 的範例程式(ch6\6-5-func9.py)。
- 函式run 的第1 個參數func 為函式物件，使用func() 呼叫執行func 所指定的函式

6-6　函式lambda

- run 函式以「run(lambda a,b: a+b, 10, 20)」執行，則func 會使用函式lambda 所定義的函式取代，函式run 內的func(x,y) 相當於函式lambda 以x 與y 為輸入參數，上述程式執行結果如下。

6-6 函式lambda

行號	範例 (：ch6\6-6-func10.py)	執行結果
1	def run(func, x, y):	k= 30
2	return func(x, y)	
3	k = run(lambda a,b: a+b, 10, 20)	
4	print('k=', k)	

程式解說

- 第1 到2 行：定義函式run，輸入三個參數func、x 與y，func 為傳入的函式物件，函式func 傳入兩個參數x 與y，回傳執行函式func 的結果。

- 第3 行：使用「run(lambda a,b: a+b, 10, 20)」相當於執行「10+20」，回傳結果儲存到變數k。
- 第4 行：使用函式print 顯示「k= 」與變數k 的值到螢幕上。

6-7 產生器(generator)

- 使用函式製作產生器，產生器可以產生一個序列的資料，產生器要使用yeild 回傳資料，而非使用return 回傳資料，使用yeild回傳資料會紀錄上一次回傳時函式的狀態，不會從頭到尾都執行。

- 使用產生器的好處是不用一次產生所有資料，當產生的資料量很大時會占用很多記憶體空間，產生器會一次產生一個資料，紀錄上一次執行的狀態，需要時再產生下一個資料。

6-7 產生器(generator)

- 還記得迴圈時使用的函式range，我們使用產生器撰寫自己的函式range，取名叫函式irange，產生器程式如右。

```
def irange(start, stop, step=1):
    if start < stop:
        i = start
        while i < stop:
            yield i
            i = i + step
    else:
        i = start
        while i > stop:
            yield i
            i = i + step
```

6-7 產生器(generator)

使用 yield 回傳資料的函式會被認爲是產生器，執行以下程式。

```
x = irange(1,10)
print(x)
```

印出以下結果。

```
<generator object irange at 0x00000000010FA360>
```

表示 x 是一個產生器 (generator) 。

以下程式使用 for 迴圈取出產生器所產生的序列元素。

```
for i in irange(1, 5, 1):
    print(i)
```

印出以下結果。

```
1
2
3
4
```

6-7 產生器(generator)

行號	範例 ( ：ch6\6-7-func11.py)	執行結果
1	def irange(start, stop, step=1):	
2	if start < stop:	
3	i = start	
4	while i < stop:	
5	yield i	<generator object irange at
6	i = i + step	0x000000000110A360>
7	else:	1
8	i = start	2
9	while i > stop:	3
10	yield i	4
11	i = i + step	4
12	x = irange(1,10)	3
13	print(x)	2
14	for i in irange(1, 5, 1):	
15	print(i)	
16	for i in irange(4, 1, -1):	
17	print(i)	

程式解說

- 第1 到11 行：定義函式irange，輸入三個參數start、stop 與step，step 沒有輸入值時，使用1 為預設值。

程式解說

- 若start 小於stop(第2 行)，表示為遞增數列，則i 設定為start(第3 行)，當i 小於stop，則使用yield 回傳變數i 的值，接著i 值遞增step(第4 到6 行)
- 否則為遞減數列，i 設定為start(第8 行)，當i 大於stop，則使用yield回傳變數i 的值，接著i 值遞增step(第9 到11 行)。

程式解說

- 第12 行：設定x 為irange(1,10)。
- 第13 行：印出x 到螢幕上。
- 第14 到15 行：使用for 迴圈取出產生器irange(1, 5, 1) 的每個元素到i, 使用函式print 印出i 的值到螢幕。

程式解說

- 第16 到17 行：使用for 迴圈取出產生器irange(4, 1, -1) 的每個元素到i, 使用函式print 印出i 的值到螢幕。

6-8 內部函式

- Python 函式內部可以包含另一個函式，函式內部的函式稱作內部函式。
- 內部函式用於函式內會一直重複利用到的功能，可以獨立出來寫成一個函式，在函式內呼叫使用，例如以下範例。

```
def hello(msg):
    def say(text):
        return 'Hello,'+text
    print(say(msg))
    print(say(' 你好'))
```

6-8 內部函式

- 函式hello 內定義函式say，在函式hello 內呼叫了兩次say，分別傳了一個字串回來，使用函式print 將字串列印出來，執行時使用以下程式呼叫函式hello。
  - hello(‘John’)
- 程式執行結果如下。
  - Hello,John
  - Hello, 你好

6-8 內部函式

行號	範例 (：ch6\6-8-func12.py)	執行結果
1	def hello(msg):	
2	def say(text):	
3	return 'Hello,'+text	Hello,John
4	print(say(msg))	Hello, 你好
5	print(say(' 你好 '))	
6	hello('John')	

程式解說

- 第1 到5 行：定義函式hello，輸入參數msg，在函式內定義內部函式say，輸入參數text，內部函式say 用於回傳「Hello,」串接參數text 的字串(第2 到3 行)。
- 呼叫內部函式say以msg為輸入，將結果印出在螢幕上，再一次呼叫內部函式say 以「你好」為輸入，將結果印出在螢幕上。
- 第6 行：呼叫函式hello 以「John」為輸入。

6-9 closure 函式

- 類似前一節的內部函式，在函式內動態建立一個函式，回傳該函式，稱作closure。
- 使用closure 的好處是可以看到外部函式的變數，讓程式碼集中在外部函式內，而非宣告定義在外部函式之外，避免程式碼分散不易閱讀。

- 將前一節程式改成closure，程式碼如下。

```
def hello(msg):
    def say(hi):
        return hi+msg
    return say
```

- 在外部函式hello 內動態建立一個函式say，函式say 會回傳參數hi(函式say 的參數) 串接參數msg(函式hello 的參數) 的字串
- 最後將函式say 當成物件回傳，內部函式say 稱作closure 函式，使用closure 函式的好處是內部函式say(closure 函式) 能夠存取外部函式hello 的參數msg。

- 呼叫函式hello 回傳函式物件say 到變數，例如以下程式碼，變數x 與y 都是函式物件。
  - x=hello('Claire')
  - y=hello('Fiona')
- 使用「x()」與「y()」執行closure 函式回傳字串，再利用函式print 列印出來。
  - print(x('Hello,'))
  - print(y('Hi,'))

執行結果

- Hello,Claire
- Hi,Fiona

6-9 closure 函式

行號	範例 ( ：ch6\6-9-func13.py)	執行結果
1	def hello(msg):	
2	def say(hi):	
3	return hi+msg	
4	return say	Hello,Claire
5	x=hello('Claire')	Hi,Fiona
6	y=hello('Fiona')	
7	print(x('Hello,'))	
8	print(y('Hi,'))	

程式解說

- 第1 到4 行：定義函式hello，輸入參數msg，在函式內定義內部函式say，輸入參數hi，內部函式say 用於回傳參數hi(函式say 的參數) 串接參數msg(函式hello 的參數) 的字串(第2 到3 行)，最後回傳函式物件say。

程式解說

- 第5 行：呼叫函式hello 以「Claire」為輸入，變數x參考到此回傳的函式物件。

程式解說

- 第6 行：呼叫函式hello 以「Fiona」為輸入，變數y參考到此回傳的函式物件。
- 第7行：呼叫函式x以「Hello,」為輸入，將回傳的字串利用函式print顯示在螢幕上。
- 第8行：呼叫函式x以「Hi,」為輸入，將回傳的字串利用函式print顯示在螢幕上。

6-10 Decorator(裝飾器)

- 裝飾器為一種函式，允許輸入一個函式，回傳另一個函式，常用於將一個自訂函式改裝成另一個函式，可以用於顯示除錯訊息，例如定義一個除錯函式如下。

```
def debug(func1):
    def func2(*args, **kwargs):
        print(' 正在執行函式 ', func1.__name__)
        print(' 函式的說明文件爲 ', func1.__doc__)
        print(' 位置引數 ', args)
        print(' 關鍵引數 ', kwargs)
        return func1(*args, **kwargs)
    return func2
```

- 函式debug 允許輸入一個函式func1，在函式debug 內定義另一個函式func2，函式func2 接收傳入函式func2 的位置引數args與關鍵字引數kwargs，顯示函式func1的函式名稱、說明文件到螢幕上

- 接著顯示傳入函式func2 的位置引數與關鍵字引數在螢幕上，回傳函式func1 以位置引數args 與關鍵字引數kwargs 為輸入的結果，最後函式debug 執行結束回傳函式func2。

- 使用上可以輸入一個函式物件到裝飾器，使用變數接收裝飾器所回傳的函式物件，輸入到裝飾器的函式名稱與接收裝飾器所回傳的函式名稱，可以使用相同函式名稱

- 如以下範例都使用add，但輸入的函式物件add 與回傳的函式物件add 是指向不同的函式物件，也就是經由裝飾器可以將函式物件轉換成另一個函式物件。

```
def add(a, b):
    ' 回傳 a 加 b 的結果 '
    return a+b
add = debug(add)
print(add(1, b=2))
```

6-10 Decorator(裝飾器)

上述程式執行結果如下。

```
正在執行函式 add
函式的說明文件爲 回傳 a 加 b 的結果
位置引數 (1,)
關鍵引數 {'b': 2}
3
```

6-10 Decorator(裝飾器)

- 也可以利用「@」簡化裝飾器的使用步驟，在需要裝飾器的函式的上一行，使用「@」串接裝飾器名稱，就可以達成使用裝飾器改變下一行所定義的函式。

```
@debug
def add(a, b, c):
    ' 回傳 a+b+c 的結果 '
    return a+b+c
print(add(1, 2, c=3))
```

- 程式執行結果如下

```
正在執行函式 add
函式的說明文件爲 回傳 a+b+c 的結果
位置引數 (1, 2)
關鍵引數 {'c': 3}
6
```

6-10 Decorator(裝飾器)

行號	範例 (：ch6\6-10-func14.py)	執行結果
1	def debug(func1):	
2	def func2(*args, **kwargs):	
3	print(' 正在執行函式 ', func1.__name__)	
4	print(' 函式的說明文件爲 ', func1.__doc__)	正在執行函式 add
5	print(' 位置引數 ', args)	函式的說明文件爲 回傳 a 加 b
6	print(' 關鍵引數 ', kwargs)	的結果
7	return func1(*args, **kwargs)	位置引數 (1,)
8	return func2	關鍵引數 {'b': 2}
9	def add(a, b):	3
10	' 回傳 a 加 b 的結果 '	正在執行函式 add
11	return a+b	函式的說明文件爲 回傳 a+b+c
12	add = debug(add)	的結果
13	print(add(1, b=2))	位置引數 (1, 2)
14	@debug	關鍵引數 {'c': 3}
15	def add(a, b, c):	6
16	' 回傳 a+b+c 的結果 '	
17	return a+b+c	
18	print(add(1, 2, c=3))	

程式解說

- 第1 到8 行：函式debug 允許輸入一個函式func1，在函式debug 內定義另一個函式func2，函式func2 接收傳入函式func2 的位置引數args 與關鍵字引數kwargs，顯示函式func1 的函式名稱、說明文件到螢幕上(第3 到4 行)

程式解說

- 接著顯示傳入函式func2 的位置引數與關鍵字引數在螢幕上(第5 到6 行)，回傳函式func1 以位置引數args 與關鍵字引數kwargs 為輸入的結果(第7 行)，最後函式debug 執行結束回傳函式func2(第8 行)。

程式解說

- 第9 到11 行：定義函式add，輸入兩個參數a 與b，增加函式的說明，回傳a 加b相加的結果。
- 第12 行：呼叫函式debug，輸入函式物件add，回傳的函式物件指定給函式物件add。

程式解說

- 第13行：函式物件add，以「1,b=2」為輸入，使用函式print將結果顯示在螢幕上。
- 第14 行：使用裝飾器debug。
- 第15 到17 行：定義函式add，輸入三個參數a、b 與c，增加函式的說明，回傳a加b加c 的結果。
- 第18行：函式物件add，以「1,2,c=3」為輸入，使用函式print將結果顯示在螢幕上。

6-11 遞迴

- 遞迴是有趣的程式設計技巧，也是一種解題策略，函式執行過程中呼叫自己，稱作「遞迴」
- 利用遞迴函式撰寫程式時，需明確定義遞迴關係，而這樣的自己呼叫自己，需要有終止的條件，若沒有終止的條件就會形成無窮遞迴。

範例6-11-1 求n 階乘 (ch6\6-11-1-fac.py)

- 我們以求解n 階乘（n!）之值為例，程式解說遞迴的觀念。數學上定義n 階乘為「n!=n*(n-1)*(n-2)*(n-3)*…*3*2*1」。

- 求解n 階乘，可以分解成n 乘以(n-1) 階乘意即「n!=n*(n-1)!」
- 求解(n-1) 階乘，可以分解成n-1 乘以(n-2) 階乘意即「(n-1)!=(n-1)*(n-2)!」
- 求解(n-2) 階乘，可以分解成n-2 乘以(n-3) 階乘意即「(n-2)!=(n-2)*(n-3)!」，依此類推

- 直到求解3 階乘，可以分解成3 乘以2 階乘意即「3!=3*2!」
- 求解2 階乘，可以分解成2 乘以1 階乘意即「2!=2*1!」
- 1 階乘(1!) 就不用再往下遞迴求解，直接就是1
- 這樣一層又一層遞迴下去，直到求解1 階乘(1!)，就終止遞迴，再一層又一層往上回推

範例6-11-1 求n 階乘 (ch6\6-11-1-fac.py)

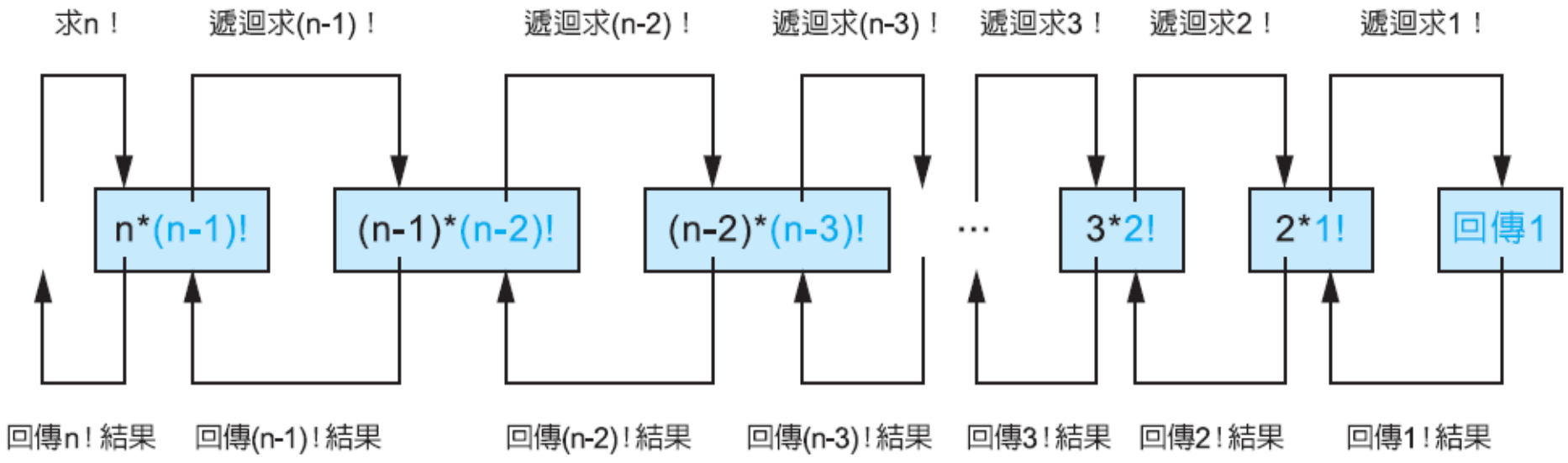


圖 6-2 求 n 階乘示意圖 1

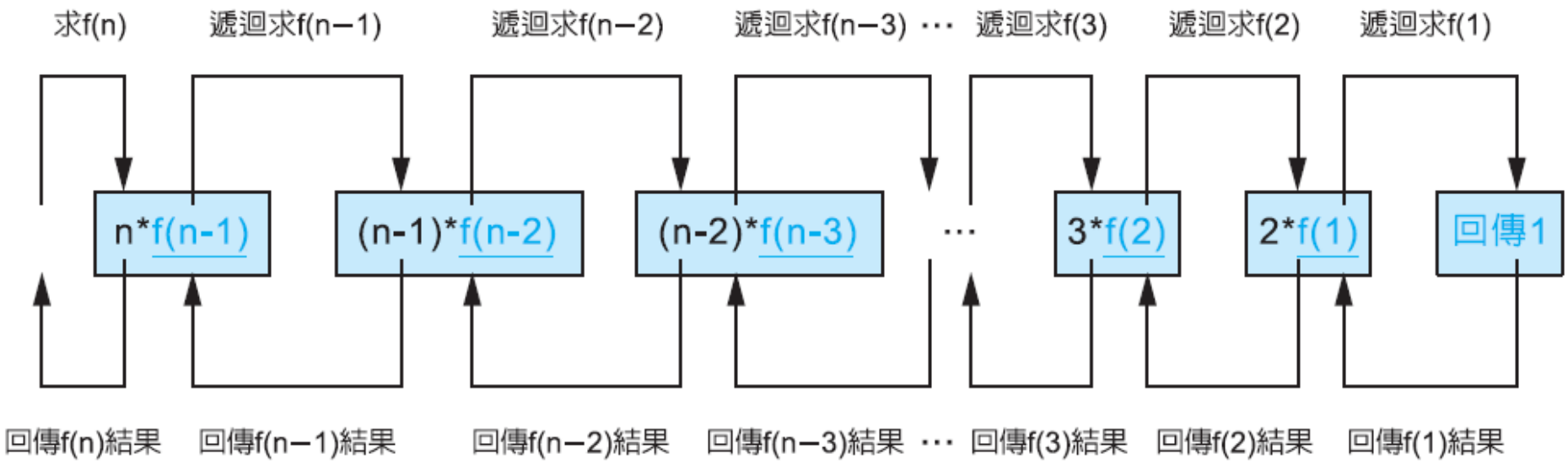


圖 6-3 求 n 階乘示意圖 2

以數學方式表達 n 階乘爲

$$f(n)=\begin{cases}1 & ,if \quad n=1\\ n*f(n-1) & ,if \quad n>1\end{cases}$$

範例6-11-1 求n 階乘 (ch6\6-11-1-fac.py)

我們接下來實作求 n 階乘的程式。

行數	程式碼 ( ：ch6\6-11-1-fac.py)	執行結果
1	def fac(num):	輸入 n 值，例如 5，程式執行結果如下。 請輸入 n 值？ 5 5! 爲 120
2	if num == 1:	
3	return 1	
4	else:	
5	return num*fac(num-1)	
6	n = int(input(' 請輸入 n 值？ '))	
7	ans = fac(n)	
8	print(n,'! 爲 ',ans,sep='')	

程式解說

- 第1 到5 行：使用遞迴函式fac 計算num 階乘，變數num 為函式fac 的輸入值，判斷num值是否符合遞迴終止條件，若num等於1，則終止遞迴，回傳1（第2 到3 行）
- 否則遞迴呼叫下去，回傳num 值乘以(num-1) 的階乘，求(num-1) 的階乘相當於遞迴呼叫函式fac，輸入參數為num-1（第4 到5 行）。

程式解說

- 第6 行：使用函式input 在螢幕輸出「請輸入n 值？」，將函式input 所回傳的字串經由函式int 轉換成整數物件，變數n參考到此整數物件。。
- 第7 行：呼叫函式fac，輸入參數n，變數ans參考到函式fac所求得的n階乘結果。
- 第8行：顯示n階乘的結果到螢幕上，使用「sep=''」將間隔設定為沒有間隔「''」。

範例6-11-1 求n 階乘 (ch6\6-11-1-fac.py)

使用圖示表示遞迴求解 5 階乘，相當於以 f(5) 執行爲例。

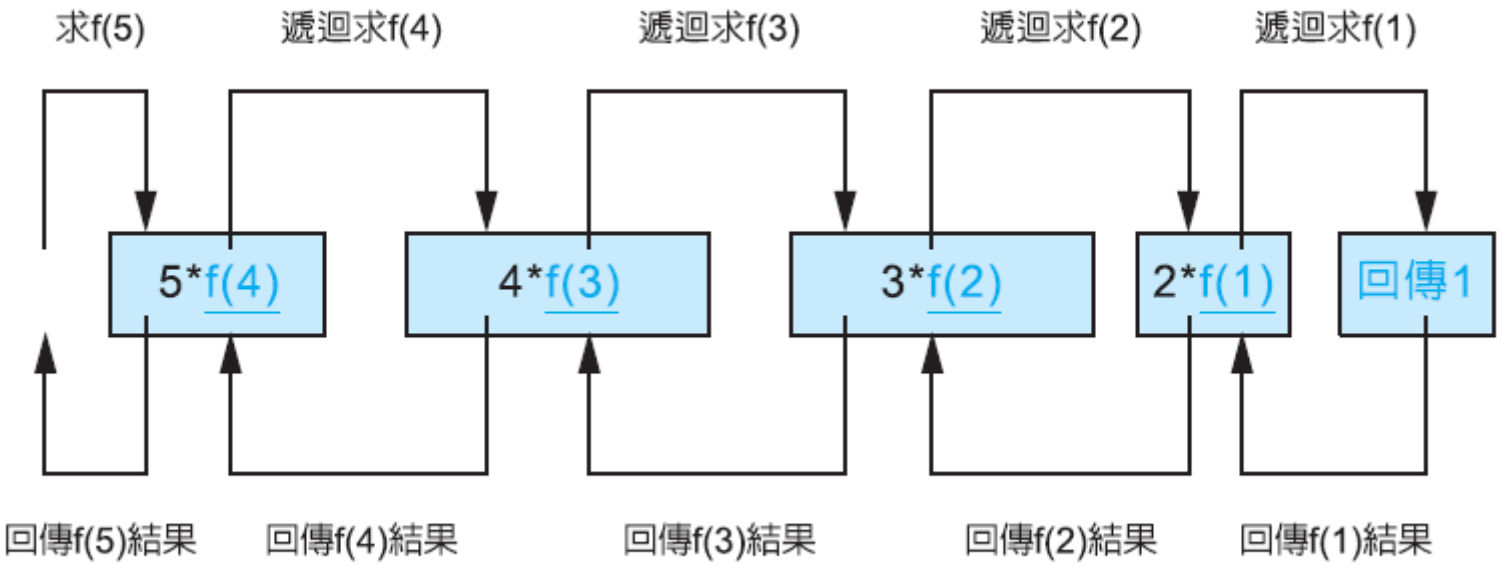


圖 6-4 求 n 階乘示意圖 3

範例6-11-2 求最大公因數 (ch6\6-11-2-gcd.py)

- 求m 與n 的最大公因數，數學上可以使用輾轉相除法求解，其原理為m 與n 的最大公因數相當於求解「n 除以m 的餘數」與m 的最大公因數，這樣一層又一層遞迴下去直到「n 除以m 的餘數」等於0，就終止遞迴，再一層又一層往上回推。

- 以求11 與25 的最大公因數為例

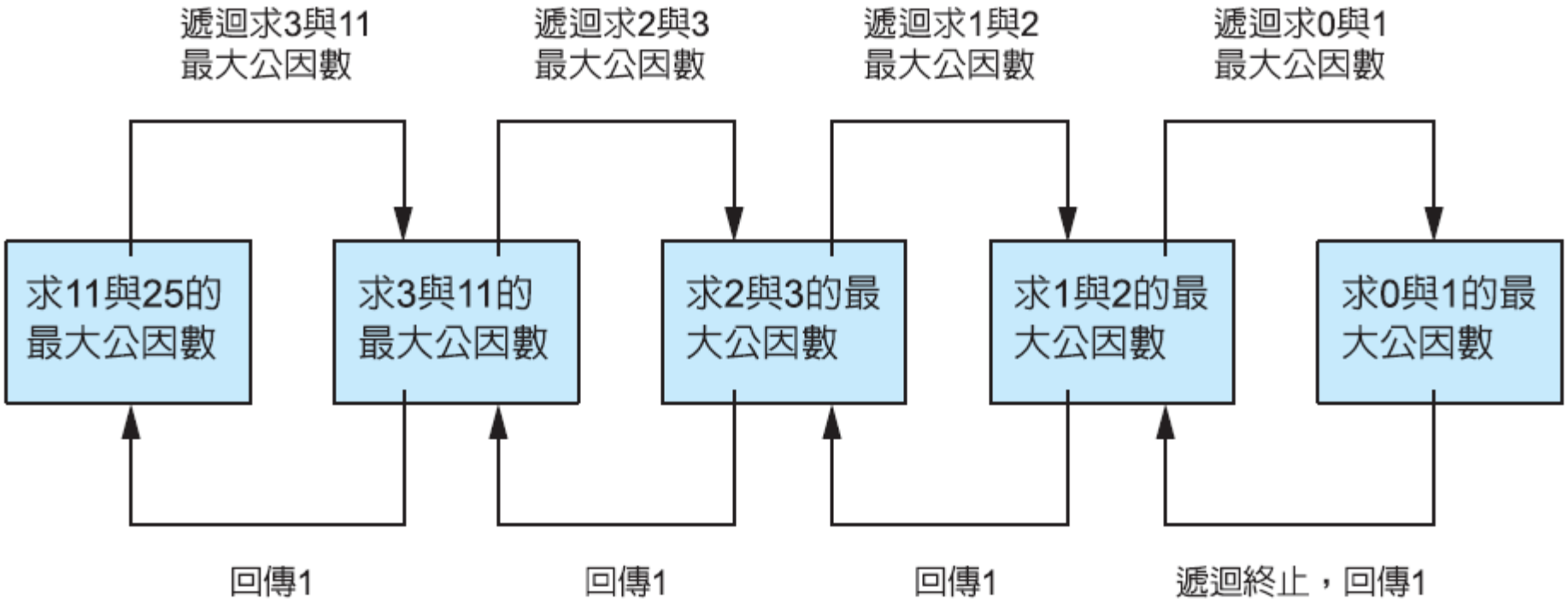


圖 6-5 求最大公因數示意圖

範例6-11-2 求最大公因數 (ch6\6-11-2-gcd.py)

以數學方式表達 m 與 n 的最大公因數爲

$$\gcd(m,n)=\begin{cases} n \quad ,if \quad m=0 \\ \gcd(n\text{除以}m\text{的餘數},m) \quad ,if \quad m\text{不等於}0 \end{cases}$$

我們接下來實作求 m 與 n 的最大公因數的程式。

( ：ch6\6-11-2-gcd.py)

行數	程式碼
1	def gcd(m, n):
2	if m == 0:
3	return n
4	else:
5	print(m, ' 與 ', n, ' 的最大公因數相當於 ', n % m, ' 與 ', m, ' 的最大公因數 ', sep='')
6	return gcd(n % m, m)
7	m = int(input(' 請輸入 m 值？ '))
8	n = int(input(' 請輸入 n 值？ '))
9	ans = gcd(m, n)
10	print(m, ' 與 ', n, ' 的最大公因數爲 ', ans, sep='')

程式解說

- 第1 到6 行：定義遞迴函式gcd 計算m 與n 的最大公因數，判斷m 值是否符合遞迴終止條件，若m 等於0，則終止遞迴，回傳變數n(第2 到3 行)
- 否則遞迴呼叫下去，顯示求最大公因數的過程(第5 行)，遞迴呼叫函式gcd，輸入參數為「n%m」與「m」(第6 行)。

程式解說

- 第7 行：使用函式input 在螢幕輸出「請輸入m 值？」，將函式input 所回傳的字串經由函式int 轉換成整數物件，變數m參考到此整數物件。
- 第8 行：使用函式input 在螢幕輸出「請輸入n 值？」，將函式input 所回傳的字串經由函式int 轉換成整數物件，變數n參考到此整數物件。

程式解說

- 第9 行：呼叫函式gcd，求m 與n 的最大公因數，變數ans參考到最大公因數。
- 第10 行：顯示m 與n 的最大公因數(ans) 在螢幕上，使用「sep=''」將間隔設定為沒有間隔「''」。

範例6-11-2 求最大公因數 (ch6\6-11-2-gcd.py)

執行結果

輸入 m 值，輸入 n 值，程式執行結果如下。

```
請輸入 m 值？ 11
請輸入 n 值？ 25
11 與 25 的最大公因數相當於 3 與 11 的最大公因數
3 與 11 的最大公因數相當於 2 與 3 的最大公因數
2 與 3 的最大公因數相當於 1 與 2 的最大公因數
1 與 2 的最大公因數相當於 0 與 1 的最大公因數
11 與 25 的最大公因數爲 1
```